

React Native & Expo Go

CCE106/L

What is React Native?

React Native is an open-source framework created by Meta (Facebook) in **2015**.

It is NOT a WebView: A common misconception. React Native does not render HTML.

The JavaScript Bridge: You write logic in JavaScript. React Native uses a "Bridge" to talk asynchronously to the phone's native UI threads.

True Native Elements: When you code a button in React Native, the Bridge tells iOS to render a ``UIButton`` and Android to render an ``android.widget.Button``.

Why React Native?

Leverage Existing Skills

Students who already know JavaScript and basic web development do not need to learn a completely new, typed language (like Swift) from scratch.

Industry Demand

React Native powers massive applications including Facebook, Instagram, Tesla, Uber Eats, and Discord. It is highly employable.

Fast Refresh

Native development requires waiting minutes for the app to recompile. React Native updates the UI instantly as you save your code, accelerating learning.

Laboratory Setup

Prerequisites

Node.js (LTS Version)

Download from nodejs.org. This installs `npm` (Node Package Manager)`, which is required to download React Native tools.

Visual Studio Code

The industry-standard code editor for JavaScript development.

Expo Go App

Have students open the Google Play Store or Apple App Store on their personal smartphones and install the free "Expo Go" application

Initializing the Project

Open the terminal in VS Code (Terminal -> New Terminal) and execute the following command. The **npx** executes a package from the npm registry without installing it globally.

```
# The command to scaffold a new blank Expo Project  
npx create-expo-app@latest HelloWorldApp --template blank
```

- **create-expo-app:** The official command-line tool to bootstrap a project.
- **HelloWorldApp:** The name of the folder that will be created.
- **--template blank:** Ensures we start with an empty canvas, without pre-built navigation, forcing students to learn the basics first.

Running the Development Server

Once the installation finishes, navigate into the newly created folder and start the Metro Bundler (the local development server).

```
# Change directory into the project folder
cd HelloWorldApp

# Start the Expo development server
npx expo start
```

A large QR code will print directly inside their terminal. Ensure that your phone and computer are on the same Wi-Fi network. Android users scan it with the Expo Go app; iOS users scan it with their native Camera app.

Analyzing App.js

Open the **App.js**. This is the entry point of the application.

```
App.js > ...
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';

// This is a functional component. Think of it as the main UI
function
export default function App() {
  return (
    <View style={styles.container}>
      <Text>Open up App.js to start working on your app!</Text>
      <StatusBar style="auto" />
    </View>
  );
}

// StyleSheet creates a style object, similar to CSS
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '■ #fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Lab Task: Customizing the UI

Modify the **App.js**.

Change the text inside the Text tags to "**Hello BSIT World!**".

In the styles object, change the backgroundColor to "**#121212**" (Dark Grey).

Add a **color: "white"** property to a new text style so it is visible on the dark background.

React Core

Components, JSX, and Props in React Native

CCE106/L

Core Components

React Native apps do not have a Document Object Model (DOM). Standard web tags like `<div>`, `<p>`, and `<span>` will cause fatal errors. We must import and use **Core Components** from the *react-native* library.

React Native	Web equivalent	Becomes natively
<code><View></code>	<code><div></code>	UIView / ViewGroup
<code><Text></code>	<code><p></code> , <code></code>	UILabel / TextView
<code><Image></code>	<code></code>	UIImageView / ImageView
<code><TextInput></code>	<code><input></code>	UITextField / EditText
<code><ScrollView></code>	scrolling div	UIScrollView / ScrollView
<code><FlatList></code>	— none —	Recycling virtualised list
<code><Pressable></code>	<code><button></code>	Touch-handling native view

The Component Tree

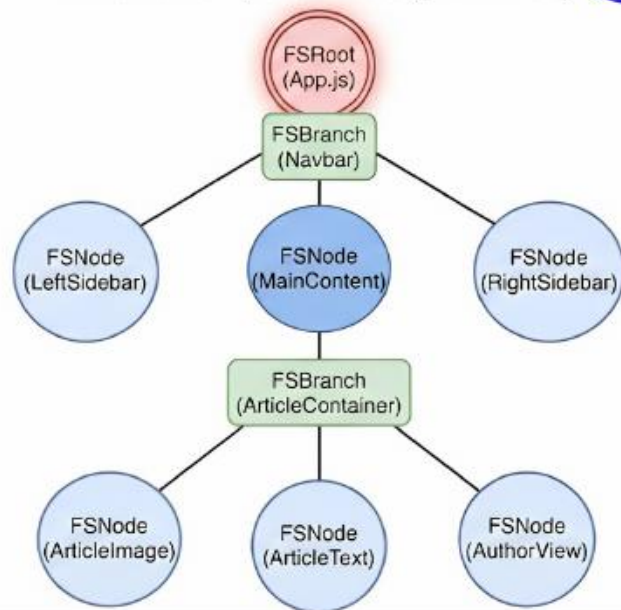
UIs in React are constructed by nesting components inside of each other, forming a tree hierarchy.

The Root: App.js acts as the root of the tree. Everything displayed must eventually be returned by this component.

Parent/Child Relationship: A component that contains a component is the parent. The text is the child.

Composition: We build complex screens by combining simple components together (e.g., combining Image, Text, and View to make a Profile Card).

A React Component Tree (Visualized)



What is JSX?

JSX is a syntax extension for JavaScript (*JavaScript XML*). It allows us to write *HTML-like markup* directly inside our JavaScript files.

Under the hood, **JSX** is translated into regular JavaScript function calls (`React.createElement()`).

The Curly Brace Rule { }

The true power of JSX is that you can embed any valid JavaScript expression inside the markup by wrapping it in curly braces.

```
// Embedding a variable
const name = "Anna";
// Creating a function that returns a string
const greeting = () => "Welcome to BSIT";
```

```
<Text>My name is {name}</Text>
<Text> 2 + 2 is {2 + 2}</Text>
<Text>{greeting()}</Text>
```

Props (Properties)

In your Java classes, you learned about Object-Oriented Programming. React Components follow a similar philosophy.

The Class: A React Component definition is like a Java Class definition. It's a blueprint for a UI element.

The Object: When you write a component tag (like `<MyComponent/>`) in JSX, React uses that blueprint to instantiate a lightweight JavaScript object (a React element or component instance).

The Constructor Arguments = PROPS: Just as you pass arguments to a Java constructor to customize an object's initial state, you pass "Props" to a React Component to customize what it displays.

Lab: Building a Profile Card

We want to build a reusable UI block for user profiles. First, we define the Component to accept a `props` argument.

1. Defining the Component

(Child)

```
import { View, Text, StyleSheet } from 'react-native';

// Function accepts 'props' object
export default function ProfileCard(props) {
  return (
    <View style={styles.card}>
      /* Accessing data via props.propertyName */
      <Text style={styles.name}>{props.name}</Text>
      <Text style={styles.role}>{props.role}</Text>
    </View>
  );
}
```

2. Using the Component

(Parent)

```
import { View } from 'react-native';
import ProfileCard from './components/ProfileCard';

export default function App() {
  return (
    <View>
      /* Passing custom data as HTML-like attributes */
      <ProfileCard name="Kimberly Arias" role="Instructor" />
      <ProfileCard name="Juan Dela Cruz" role="BSIT 4A Student" />
    </View>
  );
}
```

Lab Activity: Student Dashboard "Stat Cards"

Objectives:

- Understand how to create and export a custom React Native component.
- Learn how to pass data dynamically using Props.
- Exercise creative freedom by designing your own unique UI styles and color schemes.

Lab Activity: Student Dashboard "Stat Cards"

Part 1: Create the Component File

Let's build a reusable "Stat Card" that can display any metric on a dashboard.

1. In your project root folder, make sure you have a folder named **components**.
2. Inside components, create a new file named **StatCard.js**.
3. Type out the following base code. Notice how we accept props to make the text dynamic:

```
import { StyleSheet, Text, View } from 'react-native';

export default function StatCard(props) {
  return (
    // We apply the custom background color passed from the parent screen
    <View style={styles.card, { backgroundColor: props.bgColor }}>
      <Text style={styles.title}>{props.title}</Text>
      <Text style={styles.value}>{props.value}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  card: {
    padding: 20,
    borderRadius: 12,
    marginVertical: 10,
    width: '100%',
  },
  title: {
    fontSize: 16,
    color: '■ #ffffff',
    fontWeight: '600',
  },
  value: {
    fontSize: 28,
    color: '■ #ffffff',
    fontWeight: 'bold',
    marginTop: 5,
  },
});
```

Lab Activity: Student Dashboard "Stat Cards"

Part 2: Use Your Component in App.js

Now, let's import your custom component and display multiple instances of it.

1. Open your App.js file.
2. Replace your code with the following starter layout:

```
import { StyleSheet, Text, View } from 'react-native';
import StatCard from './components/StatCard';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.header}>My Custom Dashboard</Text>
      /* Rendering multiple cards using props */
      <StatCard
        title="Total Users"
        value="1,240"
        bgColor="#4f46e5"
      />
      <StatCard
        title="Revenue"
        value="$12,450"
        bgColor="#059669"
      />
      <StatCard
        title="Pending Issues"
        value="3"
        bgColor="#d97706"
      />
    </View>
  );
}
```

Lab Activity: Student Dashboard "Stat Cards"

Part 2: Use Your Component in App.js

Now, let's import your custom component and display multiple instances of it.

1. Open your App.js file.
2. Replace your code with the following starter layout:

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '■ #f3f4f6',
    padding: 20,
    paddingTop: 60,
  },
  header: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
    color: '□ #1f2937',
  },
});
```

Lab Activity: Student Dashboard "Stat Cards"

Part 3: The Creative Challenge

Now that the core logic is working, it's time to make this app your own. You have complete freedom to customize the design.

Fulfill at least **three (3)** of the following design challenges:

- 1. Custom Color Palette:** Change the background colors of the cards to match a specific theme.
- 2. Card Borders & Shadows:** Experiment with adding borders (`borderWidth`, `borderColor`), custom border radiuses (`borderRadius`), or shadow effects (`elevation`, `shadowColor`) in your `StatCard` stylesheet.
- 3. Add an Icon / Emoji Prop:** Add a new prop (for example, `emoji=""`) and render it inside your `StatCard` component next to the title.
- 4. Typography Adjustment:** Change font weights, text alignments (center vs. left), or text colors to make your dashboard look professional.